

PIXEL SOFTWARE DESIGN

Département de Recherche & Développement

Intelligence Artificielle Avancée

& Federated Learning

- Fondations de l'intelligence artificielle
- Algorithmes d'apprentissage fédéré
- Confidentialité, attaques et défenses
- Applications et grands modèles fédérés
- Framework FedX de référence
- Nouveaux fronts de recherche
- Travaux pratiques, projets et évaluation

Auteur	**Ghiloufi Jamel**
Fonction	**Architecte Logiciel Principal**
Direction	**Département de Recherche & Développement**
Service	**R&D - Intelligence Artificielle Avancée & Apprentissage Fédéré**
Niveau	**Recherche**
Édition	**Édition 2026 – 1.0**
Copyright	**© Pixel Software Design**

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Copyright et mentions légales

Éditeur	Pixel Software Design
Auteur	Ghiloufi Jamel
Fonction	Architecte Logiciel Principal
Département	Département de Recherche & Développement
Intitulé	Intelligence Artificielle Avancée & Federated Learning
Sous-titre	Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Édition	Édition 2026 (version 1.0)
Langue	Français
Niveau	Recherche
Collection	Supports de formation et de recherche, Pixel Software Design

Droits d'auteur

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Droits de l'auteur. Ghiloufi Jamel est l'auteur unique de l'ensemble du contenu scientifique, pédagogique et technique de cet ouvrage. Toute adaptation, traduction, reproduction ou exploitation, y compris partielle, requiert une autorisation écrite préalable.

Droits des tiers. Les algorithmes, modèles et travaux cités restent la propriété de leurs auteurs respectifs. Les références employées figurent en fin d'ouvrage, ce qui permet de remonter au travail original pour toute affirmation chiffrée.

Marques et licences externes. PyTorch, TensorFlow, Flower, Opacus, MONAI, scikit-learn et PySyft sont des logiciels libres sous licences respectives (BSD, Apache 2.0, MIT). Leur usage dans ce document est purement pédagogique et n'emporte aucune prescription commerciale.

Mentions relatives aux données et aux expérimentations

Document pédagogique à usage de formation interne. Les exemples de code fournis sont livrés à titre pédagogique ; les résultats numériques illustrés sont issus d'expérimentations conduites dans un cadre de recherche et ne constituent pas des performances de production.

Ce support synthétise les travaux de recherche du Département de Recherche & Développement de Pixel Software Design sur l'intelligence artificielle distribuée, l'apprentissage fédéré, la protection de la vie privée et les systèmes embarqués intelligents.

Sommaire

Structure détaillée du cours

Partie 1 — Ingénierie de l'IA : intégration, théorie et applications 8

1.1 Le développement et l'intégration logicielle 9

1.2 Théorie et algorithmes fondamentaux 12

1.3 Applications pratiques et cas d'usage 14

1.4 Fiche des abréviations clés 16

Annexes 19

A.1 Liste des abréviations 20

A.2 Table de notation 21

Partie	Thème	Contenu	Compétence visée	Durée
Partie 1	Fondations	Statistique, optimisation, courbes d'apprentissage	Comprendre et formaliser	2-3 semaines
Partie 2	Algorithmes FL	FedAvg, convergence, hétérogénéité, système	Implémenter et régler	3-4 semaines
Partie 3	Confidentialité	DP, attaques, défenses, agrégation sécurisée	Auditer et sécuriser	2-3 semaines
Partie 4	Applications	FedLLM, santé, V2X, FedRL, TinyML, graphes	Transposer à un domaine	3-4 semaines
Partie 5	Framework FedX	Architecture, API, benchmarks, exploitation	Industrialiser	2 semaines
Partie 6	Nouveaux fronts	FedCL, FedCI, SNN, FedGL, feuille de route	Ouvrir la recherche	2 semaines
Partie 7	TP et projets	9 labs, 5 projets, corrigés, évaluation	Faire et rendre	4-6 semaines

Préface

Comment utiliser ce cours

Ce support est conçu pour un parcours de formation et de recherche couvrant l'ensemble de la chaîne de valeur de l'intelligence artificielle distribuée : les fondations mathématiques, les algorithmes d'apprentissage fédéré, les menaces et défenses, les applications de pointe, une plateforme de référence opérationnelle, et enfin les nouveaux fronts ouverts par la recherche du Département.

Ce n'est pas un manuel de référence à lire d'un trait : c'est un **manuel opératoire**. Chaque chapitre alterne exposition, formalisation, mise en œuvre numérique et évaluation. Les listings sont complets et exécutables ; les tableaux comparatifs servent de grille de décision ; les schémas d'architecture servent de support de revue de conception.

Public et prérequis

Le contenu s'adresse à des étudiants de master et de doctorat, à des ingénieurs IA et à des praticiens de la recherche appliquée. Les prérequis sont la connaissance de l'apprentissage automatique classique, de l'algorithmique numérique et des bases de probabilités. Les parties 1 à 3 sont auto-suffisantes pour un étudiant qui découvre le domaine ; les parties 4 à 7 supposent la mécanique et les outils présentés dans la partie 2.

Conventions de lecture. Une expression mathématique est encadrée par deux signes dollars ; un identifiant, un fichier ou une commande est écrit en caractères à chasse fixe ; les encadrés *Définition*, *Théorème*, *Méthode* et *Attention* signalent les points à maîtriser et les pièges fréquents. Les renvois de la forme « voir chapitre 3.2 » désignent le second chapitre de la troisième partie.

Progression pédagogique

Brancher un modèle dans un logiciel, choisir un algorithme, livrer un cas d'usage

Partie 1

Ingénierie de l'IA : intégration, théorie et applications

Ce cours est le versant opérationnel de l'intelligence artificielle. Il suppose le cours d'introduction et traité de trois questions concrètes : comment brancher un modèle dans une application sans en devenir prisonnier, comment choisir un algorithme sans laboratoire de recherche, et comment livrer un cas d'usage qui tient debout en production.

Objectifs de la partie

- Choisir entre API du nuage, exécution locale et architecture hybride, et justifier le choix.
- Mettre en œuvre une génération augmentée par récupération, y compris ses garde-fous.
- Relier une famille d'algorithmes à une structure de données.
- Construire un cas d'usage avec un critère de succès mesurable et une porte de sortie.
- Lire et écrire les abréviations du domaine sans hésitation.

Plan de la partie

1. Le développement et l’intégration logicielle
2. Théorie et algorithmes fondamentaux
3. Applications pratiques et cas d’usage
4. Fiche des abréviations clés

Chapitre 1.1

Le développement et l'intégration logicielle

Le choix d'intégration détermine le coût, la latence, la confidentialité et la liberté de remplacement. Il se prend une fois et se paie longtemps.

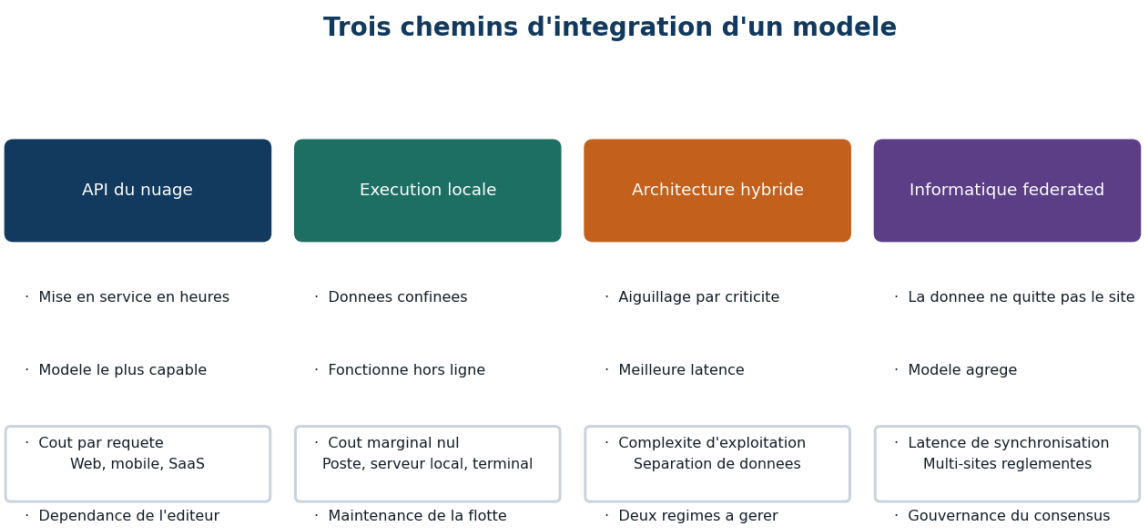


Figure 1 – Trois chemins d'intégration d'un modèle et leurs contextes d'usage

Consommer une API du nuage

C'est le chemin le plus rapide : une requête HTTP, un corps de requête en JSON, une réponse en JSON. Il permet d'utiliser un modèle très supérieur à ce que l'on pourrait entraîner, sans aucun investissement matériel.

```python
import os
import time
import httpx
from dataclasses import dataclass

# --- configuration : rien de secret dans le depot ---------------------
BASE_URL = os.environ["LLM_BASE_URL"]
API_KEY = os.environ["LLM_API_KEY"]
MODELE = os.environ.get("LLM_MODELE", "gpt-4o-mini")


@dataclass
class Reponse:
texte: str
jetons_entree: int
jetons_sortie: int
duree_s: float


def interroger(prompt: str, temperature: float = 0.0,
jetons_max: int = 800, tentatives: int = 3) -> Reponse:
"""Appel HTTP robuste : delai maximal, reprise, backoff exponentiel.

Trois defaillances sont traitees explicitement : le delai depasse, la
limitation de debit, et l'erreur transitoire du service distant.
"""
charge = {
"model": MODELE,
"messages": [{"role": "user", "content": prompt}],
"temperature": temperature,
"max_tokens": jetons_max,
}
entetes = {"Authorization": "Bearer %s" % API_KEY
"Content-Type": "application/json"}

for essai in range(tentatives):
debut = time.monotonic()
try:
r = httpx.post(BASE_URL + "/chat/completions", json=charge,
headers=entetes, timeout=30.0)
if r.status_code == 429: # trop de requetes
time.sleep(min(8.0, 0.5 * 2 ** essai))
continue
r.raise_for_status()
corps = r.json()
usage = corps.get("usage", {})
return Reponse(
texte=corps["choices"][0]["message"]["content"],
jetons_entree=usage.get("prompt_tokens", 0),
jetons_sortie=usage.get("completion_tokens", 0),
duree_s=time.monotonic() - debut,
)
except (httpx.TimeoutException, httpx.TransportError):
time.sleep(0.5 * 2 ** essai)
raise RuntimeError("service indisponible apres %d tentatives" % tentatives)
```

Listing 1 – Appel d'API robuste : delai maximal, reprise et backoff

- **Avantage :** mise en service en quelques heures, accès aux meilleurs modèles, montée en charge immédiate.
- **Inconvénient :** dépendance à l'éditeur, coût par requête, données qui sortent de l'organisation.
- **Risque principal :** la Shoulder.ApiKey dans le code. Elle se lit dans un journal d'erreur bien plus souvent qu'on ne l'imagine.

Écrire toute la logique métier autour du format de réponse d'un fournisseur rend le remplacement ultérieur très coûteux. La parade est simple et peu coûteuse : placer tous les appels derrière une interface interne unique, `Modèle` ou `Fournisseur`, et n'exposer au reste du code que trois méthodes – `completer`, `generer_structure`, `classer`.

Le piège du couplage précoce

Exécuter le modèle en local

L'exécution locale vise trois objectifs : la confidentialité des données, la continuité de service hors réseau, et l'absence de coût marginal par requête. Elle se paie par la gestion du matériel et par la mise à jour du parc.

Critère	Nuage	Local	Hybride
Délai de mise en service	heures	jours	semaines
Coût marginal	par requête	nul après amortissement	mixte
Confidentialité	dépendance de l'éditeur	totale	partielle ou totale
Fonctionnement hors ligne	non	oui	partiel
Qualité accessible	la meilleure	modérée	les deux
Charge d'exploitation	faible	élevée	élevée
Indépendance fournisseur	faible	totale	moyenne

Tableau 1 – Comparaison des modes d'exécution

Le mode hybride est rarement un compromis mou : dans la pratique, on aiguille – traitement courant en local, demande complexe au nuage, donnée sensible jamais transmise.

Architecture hybride et résilience

Une application hors ligne doit être conçue, pas négociée après coupure. La règle est que la base locale fasse autorité pour l'affichage, et que le serveur ne soit qu'une source de synchronisation.

Algorithme 1 – Synchronisation hors ligne

```
01  L'écriture se fait d'abord dans la base locale, avec un identifiant uniqueLa transaction
locale est validée immédiatement pour l'utilisateur
02  Un journal local conserve les opérations en attente de synchronisation
03  Au retour du réseau, les opérations sont rejouées dans l'ordre de leur création
04  En cas de conflit, la version du serveur l'emporte et l'écart est journalisé
05  L'interface signale a l'utilisateur ce qui n'est pas encore synchronise
```

Entrées : Journal local des opérations, état du serveur • **Sorties :** Base locale à jour, écart de synchronisation connu • **Complexité :** O(n) en nombre d'opérations en attente

La ligne 5 est la seule difficile : sans règle de conflit explicite, deux appareils convergent vers des états différents, et le bug n'apparaît que chez l'utilisateur.

Le motif de génération augmentée par récupération

C'est le motif central de l'intégration d'entreprise. Il consiste à ne jamais demander au modèle de connaître une information : on lui fournit les passages pertinents au moment de la question.

Algorithme 2 – Cycle d'une question en génération augmentée

```
01  Reformuler la question en requête de recherche documentaireRechercher les passages proches
dans la base vectorielle
02  Filtrer par droits d'accès, par date et par source autorisée
03  Assembler les passages retenus, en limitant le volume total
04  Interroger le modèle en exigeant des citations de passages
05  Vérifier que chaque affirmation est rattachee a un passage
06  Renvoyer la réponse et ses références, ou un refus explicite
```

Entrées : Question, droits d'accès de l'utilisateur, base documentaire • **Sorties :** Réponse sourcée, ou refus motivé • **Complexité :** Recherche vectorielle sous quelques dizaines de millisecondes

L'étape 7 n'est pas décorative : un système qui répond à tout, même quand il ne sait pas, est plus dangereux qu'un système qui ne répond pas.

La base vectorielle n'est pas une base de connaissances

La similarité vectorielle mesure une ressemblance de formulation, pas une pertinence factuelle. Un document sur les « taux d'intérêt » peut être retenu pour une question sur le « taux d'usure ». Le découpage des documents, l'indexation et le filtrage pèsent autant que le modèle lui-même.

Astuce

Le meilleur indicateur de disponibilité d'un système d'IA n'est pas la qualité de la réponse, c'est la **capacité à dire qu'il ne sait pas**. Mesurez le taux de refus justifiés : un système qui refuse trop est utilisable, un système qui répond toujours ne l'est pas.

A retenir

- Le mode d'intégration se décide sur la latence, la confidentialité et le budget, pas sur la qualité du modèle.
- Tous les appels doivent passer par une interface interne unique, pour rester remplaçable.
- L'hybride n'est pas un compromis mou : il se met en œuvre par aiguillage.
- La génération augmentée impose de concevoir le refus comme un comportement normal.

Chapitre 1.2

Théorie et algorithmes fondamentaux

Le choix d'un algorithme découle de la structure des données, jamais du résultat souhaité.

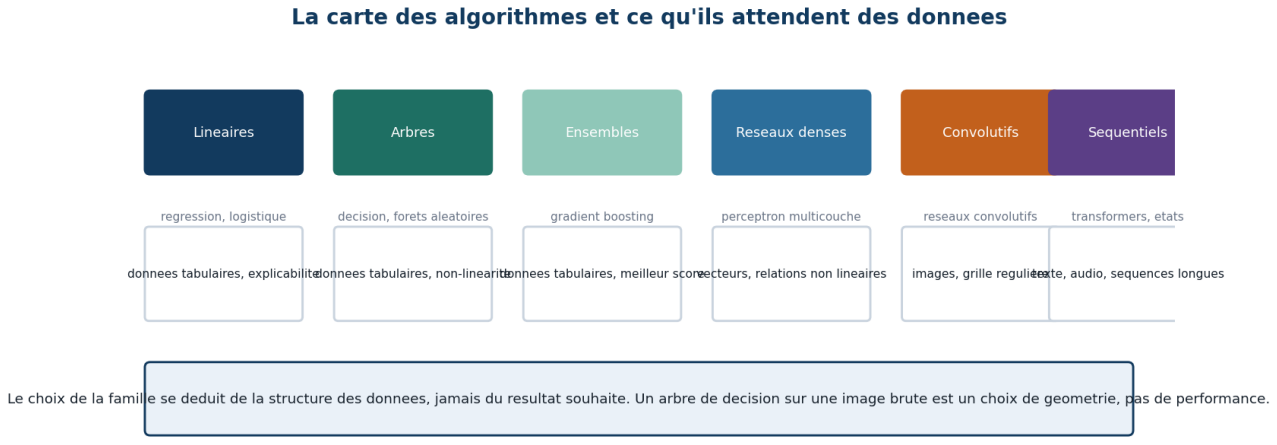


Figure 1 – La carte des algorithmes et la structure de données attendue

Régression linéaire et logistique

Ce sont les deux premiers modèles à essayer systématiquement, même sur un problème qui semble complexe. Une régression linéaire ajuste une droite ; la logistique ajuste une probabilité. Toutes les deux s'entraînent par descente de gradient.

$$w^{*} = \operatorname{argmin}_{w} (1)/(m) \sum_{i=1}^{m} \leq ft(y_i - w^{\top} x_i)^2 + \lambda \| w \|^2$$
$$P(Y=1 \mid x) = \sigma \leq ft(w^{\top} x + b), \sigma(z) = (1)/(1+e^{-z}) \quad \text{(E.1)}$$

Le terme $\lambda \| w \|^2$ est la régularisation : sans elle, le modèle explose sur des variables corrélées. C'est le premier réflexe à prendre.

- **Quand :** données tabulaires, peu de variables, besoin d'explicabilité.
- **Diagnostic :** si une régression logistique est battue par un arbre, l'interaction ou la non-linéarité compte.
- **Piège :** la multicolinéarité. Elle rend les coefficients instables sans dégrader la prédiction — ne jamais lire un coefficient comme une causalité.

Arbres de décision et forêts aléatoires

Un arbre de décision pose une suite de questions binaires : à chaque nœud, il choisit la variable et le seuil qui séparent le mieux les deux groupes restants. Une forêt aléatoire assemble des centaines d'arbres entraînés sur des tirages différents, et agrège leurs votes.

- **Force :** aucune normalisation nécessaire, gestion native des valeurs manquantes, interprétabilité directe.
- **Force :** excellent sur données tabulaires, souvent supérieur à une méthode linéaire en performance.
- **Limite :** unstable — une petite modification des données change l'arbre. D'où la forêt, qui moyenne cette instabilité.
- **Limite :** extrapolation impossible. Un arbre ne sait pas prédire au-delà de la plage observée.

> **Exemple**
> La forêt aléatoire est le premier modèle à essayer sur un jeu de données tabulaires. Sur des données textuelles ou des images, elle est en revanche nettement inférieure à un réseau de neurones : la structure de la grille ou de la séquence n'est pas représentable par des seuils sur des colonnes.

Réseaux de neurones profonds

Un réseau profond empile des transformations non linéaires. La couche dense est l'unité de base ; la convolution et l'attention sont des variantes qui exploitent une structure particulière de l'entrée.

Couche	Hypothèse sur les données	Coût dominant	Usage
Droite (MLP)	aucune	paramètresvecteurs, données tabulaires	
Convolution	grille régulière, voisinage local	calcul	images, signaux, audio
Attention	dépendance entre positions, quelle qu'elle soit	mémoire du cache	texte, audio, vidéo, code
État récurrent	dépendance strictement ordonnée	séquentialité	flux, séries temporelles

Tableau 1 – Les trois familles de couches et la structure qu'elles exploitent

L'hypothèse sur les données est le vrai choix de conception ; l'architecture n'en est que la traduction.

Ordre d'essai recommandé

- **Modèle linéaire**, comme référence et point de comparaison.

2. **Arbre ou forêt**, si les données sont tabulaires.
3. **Gradient boosting**, si le précédent est bon et qu'il reste de la performance à gagner — c'est souvent le meilleur rapport gain/complexité.
4. **Réseau profond**, si la donnée est une image, un son, un texte ou une série temporelle.
5. **Modèle de langue**, si la tâche est du langage ou du raisonnement, et qu'un appel distant est acceptable.

Un projet qui saute l'étape 1 ne peut pas démontrer que son modèle est meilleur que la solution triviale.

A retenir

- Le choix de l'algorithme découle de la structure des données, jamais du résultat attendu.
- Modèle linéaire et forêt aléatoire doivent être essayés en premier sur données tabulaires.
- Les couches denses, convolutives, d'attention et récurrentes correspondent à quatre hypothèses distinctes sur les données.
- Sans référence linéaire en point de départ, aucune amélioration n'est démontrable.

Chapitre 1.3

Applications pratiques et cas d'usage

Un cas d'usage se démontre par son coût, pas par sa démonstration.

Automatisation des processus et extraction documentaire

Le premier cas d'usage rentable reste l'extraction de données à partir de documents non structurés : factures, contrats, bulletins, reçus. La valeur est immédiate, mesurable en heures de saisie économisées, et le risque borné car la sortie est vérifiable par un opérateur.

Étape	Question	Indicateur
Cadrage	quelle décision l'extraction permet-elle ?processus concerné, périmètre réel	
Mesure	combien de documents, quelle erreur tolérée ?	volume quotidien, taux d'erreur acceptable
Évaluation	sur quels documents de référence ?	précision et rappel par champ
Déploiement	qui valide, qui corrige ?	temps humain résiduel par lot
Bilan	le gain net est-il positif ?	heures économisées moins heures de contrôle

Tableau 1 – Évaluation d'un cas d'usage d'extraction

Le dernier indicateur est le seul qui décide : sans lui, un projet d'extraction peut être un perte de temps disguisée en gain.

Analyse prédictive et logistique

Le deuxième cas mature est l'anticipation : ruptures de stock, tournées de livraison, maintenance d'équipement. La difficulté n'est pas le modèle, mais l_available data : un historique de maintenance nonexistent ou mal saisi rend la prédiction impossible, quel que soit l'algorithme.

- **Maintenance prédictive :** surveillance de signaux (vibration, température, courant) et déclenchement avant défaillance.
- **Prévision de demande :** historique de ventes, saisonnalité, événements promo_float, arbitrage coût du stock manquant contre coût du surstock.
- **Optimisation de tournées :** contraintes de livraison, fenêtres horaires, capacité des véhicules.

La prévision ne se corrige pas toute seule

Un modèle de prévision est juste en moyenne et faux aux moments qui comptent. Une erreur sur le pic de demande de décembre est incomparablement plus coûteuse qu'une erreur en mars. Il faut donc évaluer le modèle **pondéré par le coût de l'erreur**, ce qui est presque toujours absent des évaluations initiales.

Assistants spécialisés et interrogation en langage naturel

Le troisième cas est l'assistant métier capable d'interroger une base relationnelle en langage naturel. C'est un cas de figure trompeur : la démonstration est spectaculaire, la production est difficile, parce que la requête générée peut être valide en syntaxe et fausse en sens.

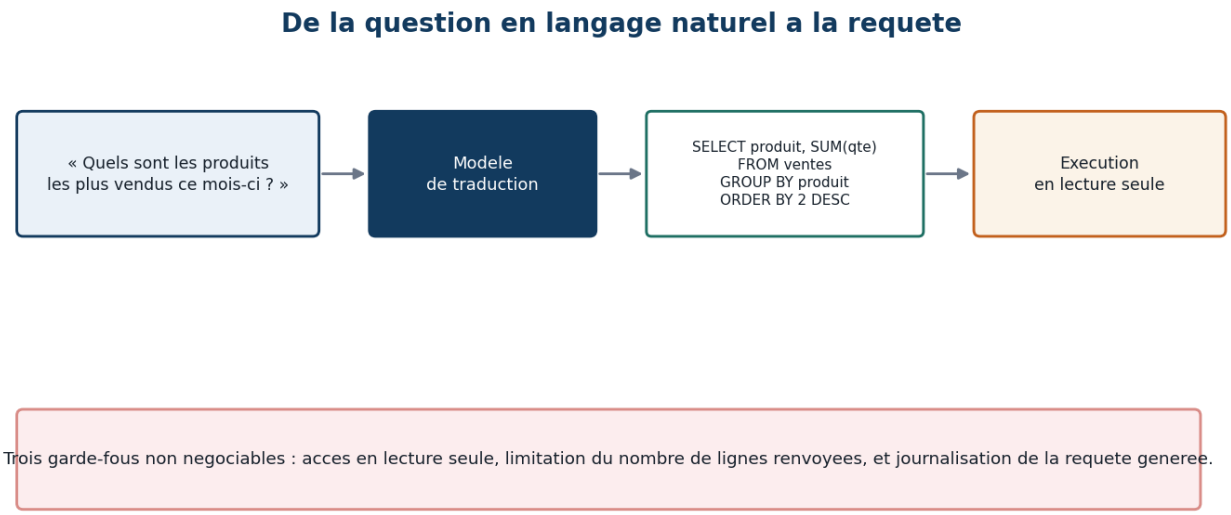


Figure 1 — De la question en langage naturel à la requête

```python
import re

MOT_CLES_INTERDITS = re.compile(
r"$\b(insert|update|delete|drop|alter|truncate|grant|revoke|$"
r"exec|execute|call|shutdown)\b", re.IGNORECASE)

REGLES = [
(r"\b(numero de|numero du|n°)\s*client\b", "numero_client"),
(r"\bprix\b", "prix_unitaire"),
(r"\b(client|clients)\b", "numero_client"),
]


def securiser(requete_sql: str, max_lignes: int = 500) -> str:
"""Valide une requete generee avant execution.

Le modele n'est pas fiable : la requete produite peut etre syntaxiquement
correcte et.sql irrelevante. Ce controle est donc obligatoire, et il
doit ecrire dans la logique, pas dans la documentation.
"""
q = requete_sql.strip().rstrip(";")
if not q.lower().startswith(("select", "with")):
raise ValueError("seules les lectures sont autorisees")
if MOT_CLES_INTERDITS.search(q):
raise ValueError("instruction non autorisee : %s" % MOT_CLES_INTERDITS.search(q).group(0))
if ";" in q:
raise ValueError("plusieurs instructions detectees")
if not re.search(r"\blimit\s+\d+", q, re.IGNORECASE):
q += " limit %d" % max_lignes
return q.replace("select *", "select *"), q
```

Listing 1 — Garde-fous d'une interrogation en langage naturel

Astuce

La règle qui évite 90 % des incidents d'un assistant de requêtage : le compte de l'utilisateur est toujours lu dans le référentiel d'autorisations, jamais déduit de ce que le modèle dit. Le modèle propose, l'autorisation tranche.

Checklist de livraison d'un cas d'usage

- **Critère de succès écrit**, mesurable, avant l'écriture du code.
2. **Jeu d'évaluation figé**, représentatif, jamais modifié après.
3. **Voie de repli** : que se passe-t-il si le modèle est indisponible ?
4. **Journalisation** de chaque appel, avec sa version de modèle et ses paramètres.
5. **Retour arrière** : une action effectuée doit pouvoir être annulée.
6. **Seuil d'abandon** : sous quel résultat on arrête le projet ?
7. **Bilan chiffré** : gain net, coût total, temps de gestion.

A retenir

- L'extraction documentaire reste le cas d'usage le plus rentable parce que sa sortie est vérifiable.
- La prévision doit être évaluée pondérée par le coût de l'erreur, pas en moyenne simple.
- Un assistant de requête en langage naturel exige un contrôle d'autorisation hors du modèle.
- Sans critère de succès écrit ni voie de repli, un projet d'IA n'est pas pilotable.

Chapitre 1.4

Fiche des abréviations clés

Trente termes qui reviennent dans tous les documents du domaine.

La maîtrise du vocabulaire abrégé est ce qui distingue un interlocuteur technique d'un lecteur non technique. Cette fiche couvre les sigles que l'on rencontre dans les spécifications, les offres d'architecture et

les comptes rendus de réunion.

Abréviation	Terme anglais	Signification
ADR	Architecture Décision Record	Trace écrite d'une décision d'architecture
AGI	Artificial General Intelligence	Intelligence artificielle générale
API	Application Programming Interface	Interface de programmation entre logiciels
BERT	Bidirectional Encoder Représentations from Transformers	Encodeur transformateur pré-entraîné
CNN	Convolutional Neural Network	Réseau de neurones convolutif, orienté image
CPython	—	Interpréteur Python de référence
CUDA	Compute Unified Device Architecture	Plateforme de calcul parallèle NVIDIA
DP	Differential Privacy	Confidentialité différentielle
FL	Federated Learning	Apprentissage fédéré
GPU	Graphics Processing Unit	Processeur graphique, accélérateur d'entraînement
KV	Key-Value	Cache d'attention, proportional à la longueur du contexte
LLM	Large Language Model	Modèle de langage de grande taille
ML	Machine Learning	Apprentissage automatique
MLP	Multi-Layer Perceptron	Perceptron multicouche, réseau dense
NLP	Natural Language Processing	Traitement automatique du langage naturel
ONNX	Open Neural Network Exchange	Format d'échange de modèles
RAG	Retrieval-Augmented Génération	Génération augmentée par récupération
RNN	Recurrent Neural Network	Réseau de neurones récurrent
SDK	Software Development Kit	Kit de développement logiciel
SSM	State Space Model	Modèle à espace d'état
SQL	Structured Query Language	Langage de requête pour bases relationnelles
SQLi	SQL Injection	Injection SQL, faille d'injection de requête
SSE	Server-Sent Events	Flux d'événements serveur à sens unique
SSL	Secure Sockets Layer	Couche de sockets sécurisée
TLS	Transport Layer Security	Sécurité de la couche transport
TPS	Tokens Per Second	Débit de génération, en jetons par seconde
VLM	Vision-Language Model	Modèle vision-langage
WebSocket	—	Connexion bidirectionnelle persistante

Tableau 1 – Fiche des abréviations clés en intelligence artificielle

Les sigles se prononcent lettre par lettre à l'anglaise : « C-N-N », pas « cé-ène-ène ». La seule exception courante est « l'API », que l'on prononce à la française.

Paire	Différence réelle	Conséquence de la confusion
IA faible / IA forte	généraliste contre spécialiséecroire qu'un modèle est « conscient »	
ML / apprentissage profond	le second est un cas du premier	sur-dimensionner un problème simple
Transformer / RNN	parallèle contre séquentiel	estimer à tort le coût d'entraînement
Exactitude / AUC	seuil Optimal contre classement	déclarer un détecteur performant sur un déséquilibre
Fuite d'information / surapprentissage	variable indisponible contre mémorisation	déployer un modèle inutilisable
LLM / modèle de langage	taille contre nature	sur-dimensionner une infrastructure

Tableau 2 – Les confusions les plus fréquentes

Astuce

La règle de lecture la plus rentable en revue technique : remplacer chaque sigle par sa définition en toutes lettres, puis relire la phrase. Neuf fois sur dix, une ambiguïté apparaît.

A retenir

- Trente abréviations suffisent à couvrir l'essentiel des documents du domaine.
- Une abréviation mal comprise produit une erreur d'architecture, pas une simple incompréhension de vocabulaire.
- Substituer mentalement la définition en toutes lettres est le meilleur réflexe de relecture.

Abréviations, notation et ressources de consultation

Annexes

Annexes

Ce cours se termine par deux annexes de référence. La première donne la signification de chaque sigle employé, sous les deux formes — sigle et développé — car la litterature emploie indifféremment l'une ou l'autre. La seconde donne la signification de chaque symbole mathématique : elle permet de relire une démonstration sans revenir au chapitre dont elle est extraite, et de vérifier qu'un même symbole garde le même sens d'un chapitre à l'autre.

Chapitre A.1

Liste des abréviations

Les sigles ci-dessous sont ceux que le cours emploie. Chacun est donné avec sa forme développée et le sens précis qu'il prend ici, qui n'est pas toujours celui qu'on lui donne ailleurs.

Un sigle employé sans être développé oblige le lecteur à le reconnaître au passage, ou à l'abandonner. Le tableau ci-dessous ferme la liste de ceux que le cours emploie : il est exhaustif pour ce document, et non pour la discipline.

Sigle	Forme développée	Sens dans ce cours
ADR	Architecture Decision Record	Fiche qui trace une décision d'architecture et sa justification.
AGI	Artificial General Intelligence	Intelligence artificielle générale.
API	Application Programming Interface	Interface d'échange entre deux logiciels.
AUC	Area Under the Curve	Aire sous la courbe de ROC, mesure synthétique du pouvoir discriminant.
BERT	Bidirectional Encoder Representations from Transformers	Modèle de représentation contextuelle pré-entraîné.
CNN	Convolutional Neural Network	Réseau convolutif.
CUDA	Compute Unified Device Architecture	Modèle de programmation des accélérateurs graphiques.
DP	Differential Privacy	Vie privée différentielle.
F1	Score F1	Moyenne harmonique de la précision et du rappel.
FL	Federated Learning	Apprentissage fédéré.
GPU	Graphics Processing Unit	Processeur de calcul massivement parallèle.
gRPC	gRPC Remote Procedure Call	Protocole d'appel de procédure distant, fondé sur HTTP/2.
JSON	JavaScript Object Notation	Format d'échange de données textuel.
KV	Key-Value	Cache clé-valeur, usuel pour l'auto-attention.
LLM	Large Language Model	Grand modèle de langage.
MLP	Multi-Layer Perceptron	Réseau de couches entièrement connectées.
NLP	Natural Language Processing	Traitement automatique des langues.
ONNX	Open Neural Network Exchange	Format d'échange de modèles entre frameworks.
RAG	Retrieval-Augmented Generation	Génération augmentée par récupération de documents.
RNN	Recurrent Neural Network	Réseau récurrent.
SSE	Server-Sent Events	Flux d'événements poussé d'un serveur vers un client.
SSL	Secure Sockets Layer	Couche de chiffrement des échanges, normalisée aujourd'hui sous le nom TLS.
TLS	Transport Layer Security	Chiffrement et authentification du canal de communication.
VLM	Vision-Language Model	Modèle qui traite l'image et le texte conjointement.

Tableau 1 – Abréviations employées dans le cours

Chapitre A.2

Table de notation

Les symboles ci-dessous gardent le même sens d'un chapitre à l'autre. Lorsqu'un chapitre doit réemployer un symbole pour un autre objet, il le redéclare sur la page même.

La notation est celle du cours : vecteurs en gras, matrices en majuscules grasses, scalaires en italique. Les espérances sont notées $\mathbb{E}$, les lois de données $\mathcal{D}$, les espaces de paramètres Θ.

Symbole	Signification
$x \in \mathbb{R}^d$	Observation de dimension d.
f_θ	Modèle paramétré par le vecteur θ.
$\theta \in \Theta$	Vecteur de paramètres, appartenant à l'espace Θ.
L	Fonction de perte.
$L(\theta)$	Valeur de la perte sur le jeu entier.
$J(\theta) = (1)/(m)\sum_{i=1}^{m} L(f_\theta(x_i), y_i)$	Objectif d'entraînement, à minimiser.
m	Nombre d'exemples du lot courant.
B	Taille du lot d'entraînement.
N	Nombre total d'exemples, données d'évaluation comprises.
n	Nombre d'exemples d'apprentissage seuls.
η	Taux d'apprentissage.
β_1	Moyenne mobile du premier moment du gradient.
β_2	Moyenne mobile du second moment du gradient.
$\square$	Terme stabilisateur des divisions, ou budget de vie privée selon le chapitre.
λ	Pondération de la régularisation.
C	Seuil de décision, en classification binaire.
p	Précision d'un détecteur.
r	Rappel d'un détecteur.
TPR	Taux de vrais positifs.
FPR	Taux de faux positifs.
Δt	Temps de réponse d'un service.

Tableau 1 – Symboles mathématiques et leur signification

> **Convention**
> Toute divergence constatée entre cette table et un chapitre signale une erreur de transcription, non une définition nouvelle : la table fait foi.

> **A retenir**
> - Un sigle employé sans être listé ici est un oubli de l'auteur, non une définition nouvelle.
> - Un symbole qui change de sens d'un chapitre à l'autre rend toute démonstration ininterprétable : c'est la principale cause des contresens que nous avons relevés.
> - Ces deux tables sont les seules parties du cours à maintenir à la main : le reste se relit tout seul.

Index des figures et des tableaux

Référence rapide du matériel visuel

Figures

1. Trois chemins d'intégration d'un modèle et leurs contextes d'usage
1. La carte des algorithmes et la structure de données attendue
1. De la question en langage naturel à la requête

Tableaux

1. Comparaison des modes d'exécution
1. Les trois familles de couches et la structure qu'elles exploitent
1. Évaluation d'un cas d'usage d'extraction
1. Fiche des abréviations clés en intelligence artificielle
2. Les confusions les plus fréquentes
1. Abréviations employées dans le cours
1. Symboles mathématiques et leur signification

Fin du document

Intelligence Artificielle Avancée & Federated Learning — Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Ghiloufi Jamel, Architecte Logiciel Principal — Pixel Software Design
Département de Recherche & Développement — Recherche
Pixel Software Design — Édition 2026

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Document généré automatiquement le 29/09/2026 – mise en page Pixel Software Design.